

Deep Learning Project Report

Messy Mashup:

Robust Music Genre Classification

Course: Introduction to DL and GenAI

By: Sahil Alam

Roll: 23f1002354

Email ID: 23f1002354@ds.study.iitm.ac.in

1 1. Abstract / Executive Summary

The “Messy-Mashup” project tackles the complex challenge of robust music genre classification. This competition requires models to categorize tracks generated by combining overlapping stems from different songs of the same genre, adjusting tempos, and injecting environmental noise. By converting audio signals into log-Melspectrogram representations and employing 2D-Convolutional Neural Networks (CNNs), I evaluated multiple architectures. I explicitly compared deep transfer-learning models (ResNet variations, EfficientNet) against custom-built CNN baselines. To bridge the massive clean-to-noisy domain gap, an extensive dynamic augmentation and preprocessing pipeline was engineered to simulate test conditions. Ultimately, the pre-trained **ResNet50_1** architecture achieved the highest generalization, proving that deep transfer learning combined with aggressive target-domain augmentation successfully isolates genre-specific musical structures despite heavy acoustic interference.

2 2. Introduction

2.1 Problem Statement

The objective of this Kaggle competition is to accurately predict the genre label (out of 10 possible classes) for highly noisy mashup tracks. The training data provides 4,000 clean, instrument-separated stems (drums, vocals, bass, other), allowing models to learn pure genre features. In contrast, the 3,020 test tracks are “mashups”—unlabeled files containing cross-song stem combination, tempo adjustments, and additive environmental noise from the ESC-50 dataset.

2.2 Project Objective

The primary goal is to build a deep learning model that accurately classifies music genres despite heavy background noise and messy audio mixing. By doing this, I aim to achieve a high Macro F1 score and a strong ranking on the Kaggle competition leaderboard. A secondary objective is to compare how well large, pre-trained image models (like ResNet) handle audio data compared to smaller, custom models built from scratch.

2.3 Report Structure

This report outlines the following components of the project:

- **Section 3: Dataset & Pre-processing** – Details the Exploratory Data Analysis (EDA) and the robust preprocessing pipeline designed to simulate test conditions.
- **Section 4: Detailed Model Architectures** – Explores the evaluated architectures, with a specific focus on the custom-built ScratchCNN and its layer-by-layer dimensionality.
- **Section 5: Loss Function & Optimizer** – Outlines the core training configurations used across models.
- **Section 6: Results & Performance Analysis** – Provides quantitative analysis, visual learning curves, and performance discussions between different freezing strategies.
- **Section 7: Conclusion** – Summarizes key learnings and critical challenges resolved during pipeline construction.

3 3. Dataset & Pre-processing

3.1 Summary of Differences Between Training and Test Sets

A detailed exploratory data analysis (EDA) revealed critical domain gaps between training stems and test mashups. These differences drove my pre-processing strategy:

- **Sampling Rate:** The training audio files have a sampling rate of 44,100 Hz, whereas the test audio files use 22,050 Hz. This difference may lead to variations in spectral representations; thus, I decided to downsample to 16,000 Hz during pre-processing.
- **Audio Duration:** Training clips are highly consistent with an average duration of 30 seconds and very low variance, while test clips show greater variability (mean 28.65 s). I decided to randomly crop 5 seconds for training to increase diversity and use a 5 second sliding window for inference.
- **Silence Proportion:** The training set contains a higher proportion of near-silent samples (16.8%), because isolated stems pause and are not continuous. The test set is a complete song containing all stems with added noise, possessing almost no silent samples (0.09%). Therefore, I decided to mix the stems from different songs during training.
- **Signal Energy (RMS):** The average RMS energy in the test set (0.128) is significantly higher than the training set (0.045), indicating that the test signals are louder because the test set comprises of complete songs containing all four stems with added noise. Thus, I normalized the waveforms during preprocessing.

3.2 Train-Validation Split

For each of the 10 genres (100 songs total), the data was split into the following:

- 80 songs → Train Set
- 20 songs → Validation Set

It was crucial to split the data *before* pre-processing to avoid data leakage, as mixing stems from different songs was one of my primary pre-processing steps.

3.3 Pre-processing Pipeline & Dataset Creation

The goal of my pre-processing was to simulate test conditions as closely as possible. For each training sample, the following pipeline was executed:

1. **Stem Selection:** I randomly selected *drums* from Song A, *bass* from Song B, *vocals* from Song C, and *others* from Song D (strictly within the same genre and train split).
2. **Down-sampling:** Down-sampled to 16,000 Hz to deal with the sampling rate mismatch.
3. **Gain/Scaling:** Randomly applied gain per stem to induce stem-level variations and increase robustness.
4. **Mixing:** Mixed the selected stems together.
5. **Cropping:** Randomly cropped a 5-second segment to decrease training time and add diversity.
6. **Noise Injection:** Added a random noise file with probability $p = 0.3$.

7. **Normalization:** Normalized the audio to deal with loudness bias.

8. **Feature Extraction:** Computed the log-melspectrogram.

The processed tensors were saved as .pt files to drastically reduce training time. The final pre-processed dataset contained **30,000 Train samples** (3,000 per genre) and **7,500 Validation samples** (750 per genre).

3.4 Inference Steps (Test Mashups)

For the inference pipeline, I loaded each mashup file, down-sampled it to 16,000 Hz, and saved the waveforms as .pt files. Spectrograms were not saved because my inference utilized a **5-second sliding window**:

1. Divide the file into segments of 5 seconds each.
2. Pad the last segment with zeros if needed.
3. Compute the log-melspectrogram for each segment.
4. Retrieve the model prediction for each segment.
5. Average the probabilities across all segments to get the final prediction.

4 4. Detailed Model Architectures

I framed the classification task as an image recognition problem, treating the log-Melspectrograms as single-channel (grayscale) images.

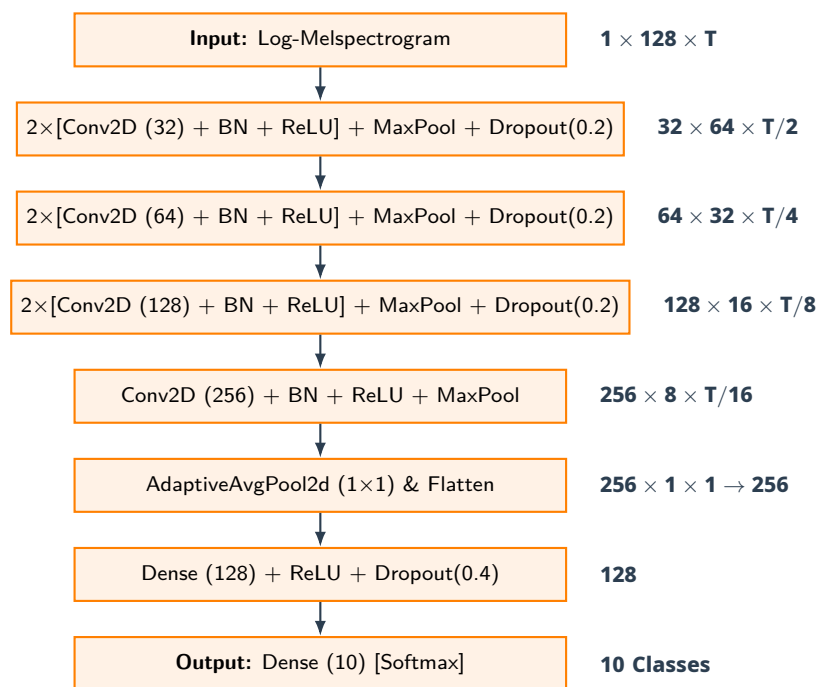
4.1 Pre-trained Networks (Transfer Learning)

- **ResNet Variations:** I established high-performance baselines using various ResNet depths and freezing strategies to analyze the impact of transfer learning depth:
 - **ResNet_1 (ResNet18):** Only layers 3, 4 and the fully connected (fc) layers were unfreezed. This approach relied heavily on early ImageNet weights to extract fundamental edges and textures while fine-tuning the deeper layers specifically for audio patterns.
 - **ResNet_2 (ResNet18):** All layers of the network were unfreezed. This allowed the entire feature extraction hierarchy to adapt perfectly to the log-Melspectrogram domain from the ground up.
 - **ResNet50_1:** Utilized deep residual blocks to capture highly complex hierarchical audio motifs with all layers active, offering the highest potential capacity.
- **EfficientNet Variations:**
 - **EfficientNet_1:** Only layers 6, 7 and the classifier were unfreezed. This offered optimized compound scaling for an efficient compute-to-accuracy ratio while preserving the foundational visual feature extractors.

4.2 Baseline Networks: ScratchCNN

To strictly evaluate the performance of models unassisted by transfer learning, I built **ScratchCNN_1** and **ScratchCNN_2** entirely from scratch. The network sequentially stacks convolution blocks. The first three blocks consist of two consecutive Conv2D and BatchNorm2d layers, followed by a MaxPool2D and a Dropout layer. The fourth block contains a single Conv2D, BatchNorm2d, and MaxPool2D. Finally, an AdaptiveAvgPool2d resolves temporal and spatial inconsistencies before flattening into a dense layer.

The diagram below illustrates the exact architectural flow and tensor dimensionalities at every layer for my **ScratchCNN** base model, assuming an input log-Melspectrogram with 128 frequency bins and a temporal dimension T :



5. Loss Function & Optimizer

The training configuration utilized the following standard approach for multi-class classification:

- **Loss Function:** `nn.CrossEntropyLoss()`
- **Optimizer:** The `torch.optim.Adam` optimizer was utilized and configured to properly filter for parameters requiring gradients (`requires_grad`).
- **Learning Rate Scheduler:** A `ReduceLROnPlateau` scheduler was utilized to dynamically manage convergence. It was configured with `mode='max'` (since I monitored the Validation Macro F1 score), `factor=0.5`, and `patience=2`.

6. Results & Performance Analysis

6.1 Evaluation Metrics

The primary metric is the **Macro F1 Score**. It computes the harmonic mean of precision and recall independently for each genre and averages them equally.

6.2 Training Performance Visualizations

The chart below visualizes the progression of the Validation Macro F1 score across training steps for different architectures.

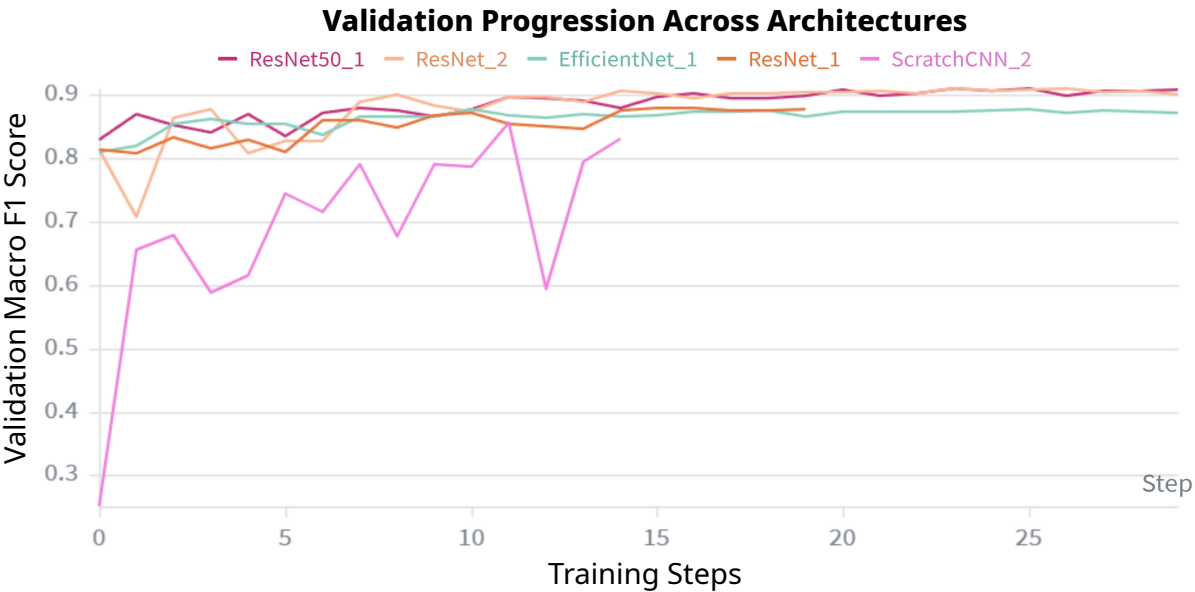


Figure 1: The ResNet variations demonstrate superior feature extraction capabilities over the custom CNN baselines.

6.3 Comparative Report & Analysis

Model	Peak Val Macro F1	Best Step	Train Acc (Step 29)	Cost Profile
ResNet50_1	0.9096	23	0.9995	High Compute
ResNet_2	0.9091	26	0.9997	High Compute
ResNet_1	0.8791	15	0.9956	Medium Compute
EfficientNet_1	0.8764	10	0.9757	Balanced/Fast
ScratchCNN_2	0.8555	11	0.9221	Lightweight

Table 1: Summary of experimented models highlighting performance characteristics.

Analysis:

- **Transfer Learning Depth & ResNet18 Freezing Strategies:** By analyzing **ResNet_1** (ResNet18 with only layers 3, 4, and fc unfrozen) against **ResNet_2** (ResNet18 with all layers unfrozen), a clear performance disparity emerged. ResNet_1 peaked at a 0.879 F1 score, making it a decent medium-compute compromise. However, ResNet_2 achieved an exceptional 0.9091 F1 score. This stark contrast definitively proves that while early ImageNet layers are useful, completely unfreezing the network allows the early convolution filters to

better adapt to the highly specific visual structures of log-Melspectrograms, yielding superior performance. Additionally, **ResNet_2 (all layers unfreezed) performed approximately similar to ResNet50_1** (0.9091 vs. 0.9096) but required comparatively lesser training time.

- **High-End Performance vs. Custom Baselines:** ResNet50 achieved the absolute highest score (0.9096) due to its massive parameter depth, but at the cost of high training time. Conversely, my custom *ScratchCNN_2* model trained extremely fast and had a very lightweight footprint but plateaued at a 0.8555 F1 score, demonstrating the severe representation penalty incurred by avoiding pre-trained weights entirely.

6.4 Kaggle Performance

To maximize leaderboard metrics, my final Kaggle submission was generated using the optimal checkpoint from **ResNet50_1 at Step 23** using the 5-second sliding window technique. This specific epoch captured the network's peak generalization state immediately prior to the onset of noise memorization.

7 7. Conclusion

7.1 Key Learnings

This project successfully highlighted the profound power of treating audio spectrograms as visual data. I learned that deep, pre-trained visual architectures (like ResNet) contain incredibly robust hierarchical feature extractors that transition seamlessly into the audio domain. Furthermore, the extensive EDA verified that a deep learning model is only as robust as its data pipeline; dynamic augmentations and strict normalizations were absolutely mandatory to bridge the vast acoustic domain gap.

7.2 Challenges Faced & Resolved

- **The Domain Gap & Feature-Space Misalignment:** A baseline model trained solely on clean isolated stems failed catastrophically when evaluated on dense test tracks. **Resolution:** I resolved this by building a rigorous preprocessing pipeline. I handled duration mismatches via random cropping, eliminated silence biases by cross-mixing stems, fixed sampling rate inequalities via downsampling, and injected ESC-50 noise.
- **Training vs. Test Domain Shift:** The most severe challenge was the fundamental mismatch between the raw input datasets. The training set consisted of clean separated stems with a high 44.1 kHz sampling rate, perfect 30-second lengths, and a significant portion (16.8%) of near-silent audio where the instruments rest. In contrast, the test set contained heavily mixed continuous mashups at a lower 22.05 kHz sampling rate, with variable lengths and significantly higher RMS energy (loudness bias). Left unaddressed, this massive structural disparity would have prevented any standard CNN from generalizing. **Resolution:** I engineered a comprehensive augmentation pipeline to aggressively simulate the test environment during training. I strictly down-sampled all audio to 16 kHz, randomly mixed varying stems to eliminate artificial silence gaps, extracted standardized 5-second crops for consistent temporal dimensionality, and applied rigid waveform normalization to neutralize the signal energy discrepancies.